
Debugging Lecture Material

Course: *Introduction to Vulnerability Research*

Author name: *Dr. Bernard Lampe*

Due date: *November 14th, 2019*

Introduction

1. The debugger is the vulnerability researcher's microscope: every other module's output (a crash, a suspicious pointer, a suspect state transition) ends up in a debugger for confirmation. This module teaches it as a research instrument, not a crash-response tool.
2. Mental model: a debugger is ptrace (Linux/BSD), Mach task ports (macOS), or the OS debug API (Windows: DebugActiveProcess, debug events loop) plus a symbol engine on top. Knowing the substrate explains every limitation: why attach changes timing, why breakpoints corrupt self-checking code, why core dumps lack live state.
3. The permission model of the substrate is the first thing to check when a debugger refuses to attach: Linux DAC plus ptrace_scope (Yama) plus SELinux; macOS requires the com.apple.security.get-task-allow entitlement or SIP disabled for a protected target; Windows requires SeDebugPrivilege or a matching integrity level.
4. The debugger is a *research instrument*, not a crash-response tool. Every other module's output arrives here for confirmation: a fuzzing crash (what is the exact faulting access?), a reversing hypothesis (does this branch really run?), an audit suspicion (does this path really exist?), and an exploit theory (does the write land where the report says?).
5. The two halves of the module: the mechanics (breakpoints, stepping, signals, cores — what the substrate actually does) and the research method (watchpoints on attacker data, breakpoint scripts as API tracers, live heap walks, remote and kernel sessions). The mechanics half is the vocabulary; the method half is the craft.

Debugger Mechanics

1. **Breakpoints:** a software breakpoint writes an architecture trap byte (x86 0xCC, ARM/ARM64: undefined-instruction or BRK encoding) over the instruction; the OS delivers SIGTRAP; the debugger restores, single-steps, re-arms. Consequences:
 - (a) In-memory code checksums see the patched byte — this is why software breakpoints happen to break integrity-checking targets (see the binary RE anti-reversing section).
 - (b) Read-only/signed code pages cannot host software breakpoints; use hardware ones.

2. **Hardware breakpoints:** CPU debug registers (x86 DR0–3; ARM64 DBGWVR/DBGBVR) match addresses without modifying code. Four per thread typically. Reserved for ROM/signed/self-modifying code, and for read/write *watchpoints*.
3. **Watchpoints:** break when an address is read or written. The single most valuable research primitive: set a watchpoint on the field you want to understand and let the program explain every writer to you. Syntax differs per debugger (`watch` in `gdb`, `watchpoint set write` in `lldb`).
4. **Stepping:** instruction vs. source step; step-over steps call-through, step-in enters. Step until a condition (breakpoint with condition, or `until`) beats manual repetition. Beware: stepping changes timing — race bugs disappear or appear.
5. **Signals:** the debugger intercepts what the process would have seen; check the `handle/process handle` policy for SIGSEGV/SIGBUS — default behavior often *stops* where a target's own crash handler should run, which changes the picture you see.
6. **Core dumps:** post-mortem full memory image. Weaker than live debugging (no breakpoints, some environment lost) but production-state-faithful and useful for triage of crashes you cannot reproduce live. macOS: symbolication interferes with core collection — disable it for honest cores; Linux: `coredumpctl`.
7. The requests and registers behind the mechanics, so that the limitations can be reasoned about:
 - (a) Software breakpoints are `PTRACE_PEEKTEXT/PTRACE_POKETEXT` plus a `SIGTRAP` (x86 `0xCC`, ARM `brk`, ARM64 `brk`); the tracer restores the original byte, rewinds the program counter, single-steps, and re-arms. That is why an integrity check that hashes the code page sees the breakpoint.
 - (b) Hardware breakpoints are debug-register comparisons (x86 DR0–DR3 with the control bits in DR7; ARM64 `DBGWVR/DBGBCR`), programmed through `PTRACE_POKEUSER`. Four per thread typically, and they do not modify code, which is why they are the only option for read-only, signed, or self-modifying code.
 - (c) Single-stepping is the trap flag (x86 `TF` in `EFLAGS`) or `PTRACE_SINGLESTEP`; `PTRACE_SYSCALL` stops on entry and exit of every syscall, and `PTRACE_SETOPTIONS` with `PTRACE_O_TRACESYSGOOD` is what distinguishes a syscall stop from a signal stop.
 - (d) Fork and thread handling is policy, not mechanism: `follow-fork-mode` and `detach-on-fork` decide which side of a fork the debugger stays with, and `set scheduler-locking on` is what keeps other threads from running while one is stepped (the reason a race bug disappears under the debugger).
8. Source-level conveniences worth naming, because they are the difference between a debugger and a disassembler: `condition` (stop only when an expression holds), `commands` (a script per breakpoint), `catch syscall/catch throw` (stop on a syscall or a C++ exception), `until/finish` (run to a line or to a return), and `set $var = ...` (convenience variables for counting).
9. Core collection is configured by `core_pattern` and `ulimit -c` on Linux, by `/cores` and `ulimit -c` on macOS, and by the Windows Error Reporting local dumps registry key on Windows. `coredumpctl` and `gcore` read or produce cores; `prctl(PR_SET_DUMPABLE)`

and the `setuid`-transition rule are why a privileged crash often produces no core at all.

10. Recording is the answer to non-determinism: `rr` records and replays with `reverse-step/reverse`. WinDbg TTD does the same on Windows, and `set record full` does it in `gdb` itself. When the bug is a race, record first and debug the recording, never the live process.

The Debugger as a Research Tool

1. **Data tracing by hand** — the dynamic analogue of the source auditing lecture's taint-following: if you can find attacker-controlled bytes in a memory dump (their magic value, a recognizable length encoding), a watchpoint on that address yields every reader/writer in execution order: this is the half-day alternative to inferring data flow statically.
2. **Breakpoint scripting**: a breakpoint whose script records the argument registers per hit and continues turns one function into a recorded API trace (`gdb: commands + printf + continue`; `lldb: breakpoint command scripts`). Count hits to find hot paths; dump arguments to reverse a wire format.
3. **Symbolic breakdown of a crash**: registers — what each meant at the faulting instruction, per the calling-convention knowledge from the binary RE module; stack backtrace with frame pointer vs. without (unwind-data quality); memory — dumping the target buffer and its neighbors at the fault.
4. **Instruction-level verification**: before pricing any exploitability claim, step the corruption: does the write land where the report says it does? Does the subsequent indirect call or dereference use a value the attacker wrote? The debugger is the ground-truth check on every theory.
5. **Heap inspection**: walk the allocator structures live (`glibc: main_arena/bins`; `jemalloc: run arenas`; `ASan builds: allocator metadata printed in the report`) to answer the groom-succeeded question live.
6. The exact commands behind each of those, because the method is only as good as the vocabulary:
 - (a) Map the layout: `info proc mappings` (or `vmmap/vmaps` in `gef/pwndbg`) turns the source auditing lecture's `/proc/self/maps` question into a live answer, with the ASLR slide visible.
 - (b) Read memory: `x/` for a raw dump, `ptype` for a recovered type, `find` for a value across a region, and `dump memory` to write a region to a file for offline work.
 - (c) Reconstruct the fault: `bt full` (locals per frame), `info registers`, `disassemble /r` (with the raw bytes, to check the reported instruction), and `frame/up/down` to move between the corruption and the crash.
 - (d) Automate the trace: a `commands` block that prints the argument registers and continues is an API tracer; a `python` block in `gdb` or an `lldb` script is how a per-hit transcript is produced at scale.

- (e) Modify the run: `call` executes a function inside the target (through `process_vm_writev`); `set` changes a variable or a register, and `return` forces an early return. These are the tools that turn a hypothesis into a test.
- 7. The discipline that makes this a research tool rather than a development tool: never trust a sanitizer report or a decompiler's guess without one debugger run that shows the same fact at instruction granularity. The debugger is the arbiter for every claim the other modules make.

Remote and Embedded Debugging

1. `gdbserver` / `lldb-server`: run the stub on the target, connect from the host with a matching-architecture-aware `gdb` (`gdb-multiarch`); serializes over TCP/serial. The Android module (Questions 2–3) walks the Android form of this and its failure modes (ARM/Thumb mode selection, version pairing).
2. Embedded: JTAG/SWD probes (OpenOCD, J-Link, Lauterbach) debug *before the OS boots* — the only way into a boot ROM or bootloader (ties to the binary RE bootloader section). Halting a core mid-boot to break a signature check is a standard move.
3. Kernel debugging: `kgdb/KDB` over serial, `QEMU+-s -S` for full system control through the same GDB remote protocol. Starve for symbols but you have `vm-linux`; kernel modules need `add-symbol-file` at their load addresses.
4. Practical rules: match stub/host versions; expect ARM/Thumb mode ambiguity on 32-bit ARM (LSB of addresses); account for the debugger's perturbation on target timing (network daemons slow measurably under attach).
5. The remote protocol itself is worth knowing, because every remote session is a small protocol implementation: `qXfer` transfers memory, features, and the target description; `qSupported` negotiates what the stub can do; `m` reads and `M` writes memory; `g/G` move registers; and `Z/z` set and clear breakpoints. A version mismatch between stub and host shows up as an unsupported packet, a truncated register set, or Remote 'g' packet reply is too long.
6. Kernel debugging is the same protocol against a kernel stub: `kgdb` over a serial port, or `QEMU` with `-s -S` (`gdbstub` on a TCP port, CPU halted at reset) for a full-system target. Symbols come from `vm-linux`, and each module needs `add-symbol-file` at its load address.
7. Embedded targets without an operating system are debugged through the chip's own debug port: JTAG or SWD with a probe (OpenOCD, J-Link, Lauterbach). This is the only route into a boot ROM or a bootloader (the binary RE module's boot-chain section), and halting the core mid-boot to step past a signature check is a standard move.
8. The Android form of the same recipe is the `gdbserver` walk-through in the Android module (Questions 2–3), with its own failure modes: ARM/Thumb mode selection, version pairing, missing symbols, and SELinux denying the attach.

9. Emulated targets (QEMU, Qiling) are the fallback when hardware access is unavailable: full control over the memory image, but no real device behaviour, which is the same trade the Android module's emulation question makes.

Tools Overview

1. gdb/lldb: the general-purpose pair. gdb strengths: remote, scripting (Python), kernel/embedded; lldb strengths: macOS/iOS first-class, better C++ expression eval, LLDB API.
2. Windows: WinDbg + kd (kernel), x64dbg (userland, modern UI), Application Verifier/gflags for heap plumbing. Debug engine owns the debug-event loop; expect a different mental model from gdb.
3. Time-travel debuggers: rr (Linux, record/replay with deterministic re-execution — the answer to non-deterministic race bugs the static lectures warn about), Undo, WinDbg TTD. Reverse-continue (reverse-step) turns “where did this value come from” into a bounded search.
4. Tracers as poor-man debuggers: strace/dtrace/dtruss and Frida for hook-level visibility without full stop-state control.
5. The gdb front-ends are what make it pleasant: gef, pwndbg and peda add the heap view, the memory map, and the exploit helpers; gdb-dashboard adds the layout; and gdb scripting (Python) is the extension point. Choose one and learn its commands, because the vocabulary above is the same in all of them.
6. Debuggers as part of a reversing tool: IDA and Binary Ninja both embed a debugger, so the static hypothesis (the decompiler) and the dynamic check (the debugger) live in one place. That is the most efficient arrangement for a reversing session.
7. System-wide views worth knowing: perf for sampling profiles, bpftrace/eBPF for kernel-level tracing, valgrind (memcheck, callgrind) for memory and profile instrumentation, and rr for record/replay. Each answers a question the breakpoint debugger answers slowly.
8. Symbol and core tooling: addr2line, llvm-symbolizer, ndk-stack and symbolicatecrash turn an address or a tombstone into a name; gcore/coredumpctl capture cores; objdump/otool/dumpbin read the binary the debugger is attached to.

Case Study: Debugging the Course's Own Bug

1. The fuzzing lab target's overflow (lab_target.c:46), debugged rather than ASan-report-observed:
 - (a) Break at lab_parse; run ./lab_target_asan seed_over; watch payload_len collected into registers per the calling convention.
 - (b) Step to the memcpy: the length argument is 300 while the destination is 64 — the call itself shows the bug before any report does.

- (c) Watchpoint on the destination's redzone address (ASan builds) shows exactly the first OOB write.
 - (d) Same session on the heap.c UAF (source auditing lab): watch the freed pointer's buf field; the second DELETE's strcmp reads it — reproducing the ASan finding at instruction granularity.
2. Discipline: for every sanitizer report, spend ten minutes showing the same fact in a plain debugger once. That skill is what separates bug reporters from bug confirmers.
 3. The same case study with the tools named: `gdb --args ./lab_target_asan seed_over, break lab_parse, run, x/4gx $rsi` (the length in the second argument register on x86-64 SysV), then stepi to the memcpy call. On ARM64 the same arguments are in x0/x1, which is why the calling-convention section of the binary RE module is a prerequisite.
 4. Turning it into a written understanding: record the register values, the destination buffer size (64), and the actual copy length (the seed_over payload length, 309 bytes) in the lab report. The debugger session is the evidence; the report is the deliverable.

Conclusion

Debugging converts static hypotheses into observed facts. The craft items here (watchpoints on attacker data, breakpoint scripts as API tracers, live heap walks, version-matched remote stubs) are what make the other modules' theory stick. Treatment of the same skill on Windows/kernel/embedded substrate differs only in the substrate tax; the method is constant.

The module's one rule: the debugger is the arbiter. A fuzzing crash, a reversing hypothesis, an audit suspicion, and an exploit theory are all claims until a debugger run shows the same fact at instruction granularity. A finding that cannot be shown in a debugger is a finding that cannot be priced.

References

1. GDB documentation (sourceware.org/gdb/current/onlinedocs/gdb)
2. LLDB documentation (lldb.llvm.org)
3. rr project: rr-project.org
4. OpenOCD: openocd.org
5. android_questions.txt in this repo (remote gdbserver drill, Questions 2–3)
6. Windows debugging: [docs.microsoft.com windbg](https://docs.microsoft.com/windows/debugging)
7. GDB remote serial protocol (sourceware.org/gdb/current/onlinedocs/gdb/Remote-Protocol.html)
8. Practical Malware Analysis, Sikorski & Stolz (dynamic analysis chapters)